ARM Assembler

Logic

For Boolean Operation
False: 0 (0x00000000)
True: -1 (0xFFFFFFFF)

`AND`⟨*cc*⟩⟨*S*⟩ Rd, Rn, ⟨*op1*⟩
⟨*cc*⟩: ALU ← Rn ∧ ⟨*op1*⟩
⟨*cc*⟩: Rd ← ALU
⟨*S*⟩⟨*cc*⟩: CSPR ← ALU(Flags)

`ORR`⟨*cc*⟩⟨*S*⟩ Rd, Rn, ⟨*op1*⟩
⟨*cc*⟩: ALU ← Rn ∨ ⟨*op1*⟩
⟨*cc*⟩: Rd ← ALU
⟨*S*⟩⟨*cc*⟩: CPSR ← ALU(Flags)

`MVN`⟨*cc*⟩⟨*S*⟩ Rd, ⟨*op1*⟩
⟨*cc*⟩: ALU ← $\overline{\langle op1 \rangle}$
⟨*cc*⟩: Rd ← ALU
⟨*S*⟩⟨*cc*⟩: CSPR ← ALU(Flags)

`EOR`⟨*cc*⟩⟨*S*⟩ Rd, Rn, ⟨*op1*⟩
⟨*cc*⟩: ALU ← Rn ⊕ ⟨*op1*⟩
⟨*cc*⟩: Rd ← ALU
⟨*S*⟩⟨*cc*⟩: CPSR ← ALU(Flags)

Use logical operators to manipulate bits within a value

`AND`	And	Clear bits down to zero	∧
`ORR`	Or	Set bits to one	∨
`EOR`	Exclusive Or	Toggle bits	⊕
`MVN`	Not	Toggle all bits	$\overline{\text{x}}$

In the following R1 has the value 00111100 (&3C)

Instruction	*Operation*
`MOV r1, #&3C`	
`AND r0, r1, #&0F`	00111100 ∧ 00001111 = 00001100 (&0C)
`ORR r0, r1, #&0F`	00111100 ∨ 00001111 = 00111111 (&3F)
`EOR r0, r1, #&0F`	00111100 ⊕ 00001111 = 00110011 (&33)
`MVN r0, r1`	$\overline{00111100}$ = 11000011 (&C3)

Program: nibble.s

```
 1.  ; Disassemble a byte into its high and low order nibbles
 7. Main
 8.          LDR     R1, Value           ; Load value to be disassembled
 9.          LDR     R2, Mask            ; Load the bitmask
10.          MOV     R3, R1, LSR #0x4    ; Copy high order nibble into R3
11.          MOV     R3, R3, LSL #0x8    ; Now left shift it one byte
12.          AND     R1, R1, R2          ; AND number with bitmask
13.          ADD     R1, R1, R3          ; Add the result of that to
14.                                      ; What we moved into R3
15.          STR     R1, Result          ; Store the result
16.          SWI     &11
17.
18. Value    DCB     &5F                 ; Value to be shifted
19.          ALIGN
20. Mask     DCW     &000F               ; Bitmask = %...0001111
21.          ALIGN
22. Result   DCD     0                   ; Space to store result
```


Comparison

- Compare two values, setting the flags accordingly
- Only the flags are changed, value of Rs does not change
- Compare two values: difference (effects N, Z, V, and C flags)
  CMP⟨*cc*⟩ Rn, ⟨*op1*⟩ ⟨*cc*⟩: ALU ← Rn − ⟨*op1*⟩
  ⟨*cc*⟩: CSPR ← ALU(Flags)
- Test Equal: are two values equal (only effects N and Z flags)
  TEQ⟨*cc*⟩ Rn, ⟨*op1*⟩ ⟨*cc*⟩: ALU ← Rn ⊕ ⟨*op1*⟩
  ⟨*cc*⟩: CSPR ← ALU(Flags)
- Test Bits: are specific bits set (only effects N and Z flags)
  TST⟨*cc*⟩ Rn, ⟨*op1*⟩ ⟨*cc*⟩: ALU ← Rn ∧ ⟨*op1*⟩
  ⟨*cc*⟩: CSPR ← ALU(Flags)

Flow Control

- Change location of next instruction
  relative to the current instruction
- Assembler calculates the offset from target for us
- Target may be ±32MB from current instruction
- Unconditional Branch
  BAL ⟨*label*⟩ PC ← PC + 8 + IR(offset)
- Conditional Branch
  B⟨*cc*⟩ ⟨*label*⟩ ⟨*cc*⟩: PC ← PC + 8 + IR(offset)

Condition Codes: ⟨cc⟩

General			
AL	*Al*ways		*Al*ways
CS	*C*arry *S*et	CC	*C*arry *C*lear
EQ	*Eq*ual (Zero Set)	NE	*N*ot *E*qual (Zero Clear)
VS	O*v*erflow *S*et	VC	O*v*erflow *C*lear

Signed Numbers			
GT	*G*reater *T*han	LT	*L*ess *T*han
GE	*G*reater Than or *E*qual	LE	*L*ess Than or *E*qual
PL	*Pl*us (Positive)	MI	*Mi*nus (Negative)

Unsigned Numbers			
HI	*Hi*gher Than	LO	*Lo*wer Than (aka CC)
HS	*H*igher or *S*ame (aka CS)	LS	*L*ower or *S*ame

Program: bigger.s

```
 1.  ; Find the larger of two numbers
 2.
 7.  Main
 8.          LDR     R1, Value1  ; Load the first value to be compared
 9.          LDR     R2, Value2  ; Load the second value to be compared
10.          CMP     R1, R2      ; Compare them
11.          BHI     Done        ; If R1 contains the highest
12.          MOV     R1, R2      ; otherwise overwrite R1
13.  Done
14.          STR     R1, Result  ; Store the result
15.          SWI     &11
16.
17.  Value1 DCD      &12345678 ; Value to be compared
18.  Value2 DCD      &87654321 ; Value to be compared
19.  Result  DCD     0         ; Space to store result
```


Advanced Arithmetic

- Used to calculate values larger than 32-Bits
- Split value into 32-bit sections
  Start with the least signification section and work up to the most signification section, using the Carry to bridge sections
- Add with Carry

  ADC⟨*cc*⟩⟨*S*⟩ Rd, Rn, ⟨*op1*⟩

  ⟨*cc*⟩: ALU ← Rn + ⟨*op1*⟩ + CSPR(C)
  ⟨*cc*⟩: Rd ← ALU
  ⟨*S*⟩⟨*cc*⟩: CSPR ← ALU(Flags)
- Subtract with Carry

  SBC⟨*cc*⟩⟨*S*⟩ Rd, Rn, ⟨*op1*⟩

  ⟨*cc*⟩: ALU ← (Rn − ⟨*op1*⟩) − CSPR(C)
  ⟨*cc*⟩: Rd ← ALU
  ⟨*S*⟩⟨*cc*⟩: CSPR ← ALU(Flags)

Program: add64.s

```
 7.  Main
 8.          LDR     R0, =Value1 ; Pointer to first value
 9.          LDR     R1, [R0]    ; Load first part of value1
10.          LDR     R2, [R0, #4] ; Load lower part of value1
11.          LDR     R0, =Value2 ; Pointer to second value
12.          LDR     R3, [R0]    ; Load upper part of value2
13.          LDR     R4, [R0, #4] ; Load lower part of value2
14.          ADDS    R6, R2, R4  ; Add lower 4 bytes and set carry flag
15.          ADC     R5, R1, R3  ; Add upper 4 bytes including carry
16.          LDR     R0, =Result ; Pointer to Result
17.          STR     R5, [R0]    ; Store upper part of result
18.          STR     R6, [R0, #4] ; Store lower part of result
19.          SWI     &11
20.
21.  Value1 DCD     &12A2E640, &F2100123 ; Value to be added
22.  Value2 DCD     &001019BF, &40023F51 ; Value to be added
23.  Result  DCD     0           ; Space to store result
```


Program: factorial.s

```
 4.          AREA    Program, CODE, READONLY
 7.  Main
 8.          LDR     R0, =DataTable    ; Load address of lookup table
 9.          LDR     R1, Value         ; Offset of value to be looked up
10.          MOV     R1, R1, LSL #0x2 ; Read value from table
13.          ADD     R0, R0, R1        ; index into table at R0
14.          LDR     R2, [R0]          ; Read value from table entry R1
15.          LDR     R3, =Result       ; Load address of result
16.          STR     R2, [R3]          ; Store the answer
18.          SWI     &11
19.
20.          AREA    DataTable, DATA
22.          DCD     1 ;0! = 1         ; Table containing factorials
23.          DCD     1 ;1! = 1
24.          DCD     2 ;2! = 2
30.  Value   DCB     5
32.  Result  DCW     0
```